

۱. اصل وارونگی وابستگی

یک اصل طراحی است. این اصل می‌گوید کلاس‌های سطح بالا (منطق اصلی برنامه) نباید به کلاس‌های سطح پایین (جزئیات اجرایی مثل دیتابیس) وابسته باشند؛ بلکه هر دو باید به انتزاع‌ها (مثلاً اینترفیس‌ها) وابسته باشند. این اصل به ما می‌گوید معماری کد چطور باید باشد، اما ابزار یا روش پیاده‌سازی آن را مشخص نمی‌کند.

۲. وارونگی کنترل

یک مفهوم یا دیدگاه کلی در طراحی است. در حالت سنتی، برنامه شما کنترل جریان کار و ساخت اشیاء را به دست دارد. در IoC، این کنترل وارونه می‌شود و به یک فریمورک یا سیستم بیرونی سپرده می‌شود. به زبان ساده: «تو به من زنگ زن، من خودم هر وقت لازم شد بهت زنگ می‌زنم» (قانون هالیوود). IoC فراتر از ساخت اشیاء است و در فریمورک‌های UI یا تست هم کاربرد دارد.

۳. تزریق وابستگی

یک تکنیک و الگوی طراحی برای پیاده‌سازی IoC است. وقتی کلاسی برای انجام کارش به کلاس دیگری نیاز دارد، خودش آن را با دستور new نمی‌سازد؛ بلکه آن وابستگی از بیرون (معمولاً از طریق سازنده یا کانستراکتور) به داخل کلاس تزریق می‌شود.

۴. ظرف وارونگی کنترل

یک ابزار یا فریمورک آماده است که وظیفه مدیریت خودکار فرآیند IoC و DI را بر عهده دارد. شما به این ظرف معرفی می‌کنید که کدام اینترفیس به کدام کلاس متصل است. سپس هر زمان که یک شیء درخواست کنید، این ظرف خودش به صورت خودکار زنجیره‌ای از وابستگی‌ها را می‌سازد، تزریق می‌کند و در نهایت آن‌ها را از بین می‌برد.

۵. کارخانه به عنوان ظرف

یک رویکرد دستی یا اولیه برای مدیریت وابستگی‌ها است. الگوی کارخانه (Factory Pattern) متدهایی برای ساخت اشیاء ارائه می‌دهد. وقتی پروژه بزرگ می‌شود و نمی‌خواهید از یک IoC Container آماده و پیچیده استفاده کنید، یک کلاس کارخانه مرکزی می‌سازید که وظیفه ساخت و تزریق دستی اشیاء را بر عهده می‌گیرد؛ یعنی این کارخانه نقش یک IoC Container ساده و دست‌نویس را بازی می‌کند.

۶. طول عمر سرویس‌ها در داتانت

وقتی در داتانت سرویسی را درون IoC Container داخلی آن ثبت می‌کنید، باید مشخص کنید که این شیء چه مدت باید زنده بماند. داتانت ۳ نوع طول عمر اصلی دارد:

- **Transient (گذرا):** هر بار که برنامه به این سرویس نیاز داشته باشد، یک نمونه (Instance) کاملاً جدید و نو از آن ساخته می‌شود.
- **Scoped (محدود به درخواست):** در طول یک درخواست وب (Web Request)، یک نمونه واحد ساخته شده و به تمام بخش‌ها داده می‌شود. با اتمام آن درخواست، شیء هم از بین می‌رود.
- **Singleton (تک‌نمونه):** فقط و فقط یک نمونه از شیء در کل طول عمر اجرای برنامه ساخته می‌شود و همه درخواست‌ها از همان یک نمونه استفاده می‌کنند.

ارتباط این مفاهیم با یکدیگر

ارتباط این مفاهیم را می‌توان مثل یک زنجیره از هدف نهایی تا ابزار و جزئیات اجرا دید:

1. **پایه و هدف (DIP):** همه چیز با DIP شروع می‌شود. این اصل به ما هدف را نشان می‌دهد: «کلاس‌ها را به هم وابسته نکن، آن‌ها را به اینترفیس‌ها وصل کن».
2. **استراتژی (IoC):** برای رسیدن به این هدف، از تفکر IoC کمک می‌گیریم. کنترل ساخت اشیاء را از داخل کلاس‌ها می‌گیریم و به بیرون منتقل می‌کنیم.
3. **تاکتیک اجرایی (DI):** برای اینکه عملاً کنترل را وارونه کنیم، از تکنیک DI استفاده می‌کنیم؛ یعنی به جای new کردن کدهای سطح پایین در کلاس سطح بالا، آن‌ها را از طریق سازنده به درون کلاس تزریق می‌کنیم.
4. **اتوماسیون (IoC Container یا Factory):** حالا اگر پروژه بزرگ شود، تزریق دستی تمام این اشیاء کار سختی است. اینجاست که به یک مدیریت‌کننده نیاز داریم. در پروژه‌های کوچک یا سنتی، یک Factory به صورت دستی این کار را انجام می‌دهد، اما در پروژه‌های مدرن، یک IoC Container هوشمند وسط می‌آید تا کل فرآیند ساخت و تزریق را کاملاً خودکار کند.
5. **مدیریت مصرف حافظه (Service Lifetimes):** در نهایت، وقتی IoC Container وظیفه ساخت خودکار اشیاء را بر عهده گرفت، باید بداند هر شیء را کی بسازد و کی نابود کند؛ اینجاست که Service Lifetimes (مثل Singleton یا Transient) به عنوان دستورالعمل مصرف حافظه به ظرف IoC داده می‌شود.